

1.Jdbc入门

1.1 Jdbc概述

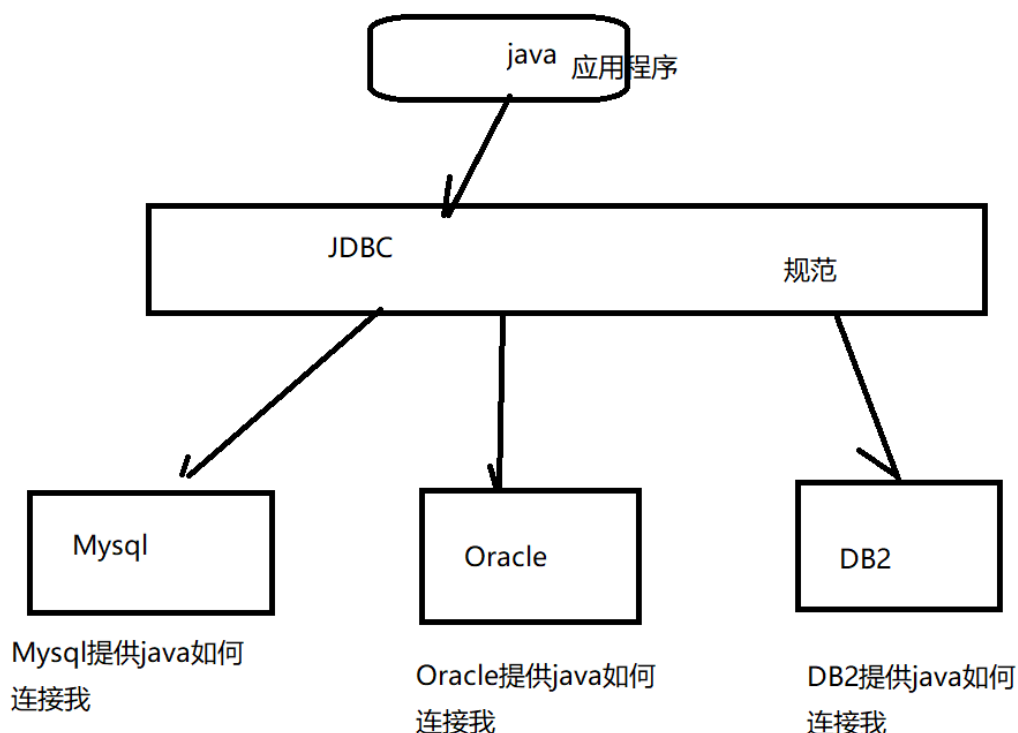
1.1.1 什么是jdbc

JDBC (Java DataBase Connectivity,java数据库连接) 是一种用于执行SQL语句的Java API。JDBC是Java访问数据库

的标准规范，可以为不同的关系型数据库提供统一访问，它由一组用Java语言编写的接口(大部分)和类组成。

JDBC是一套java连接数据库的规范（统一不同数据库的连接）

java连接数据库的一种规范（标准）



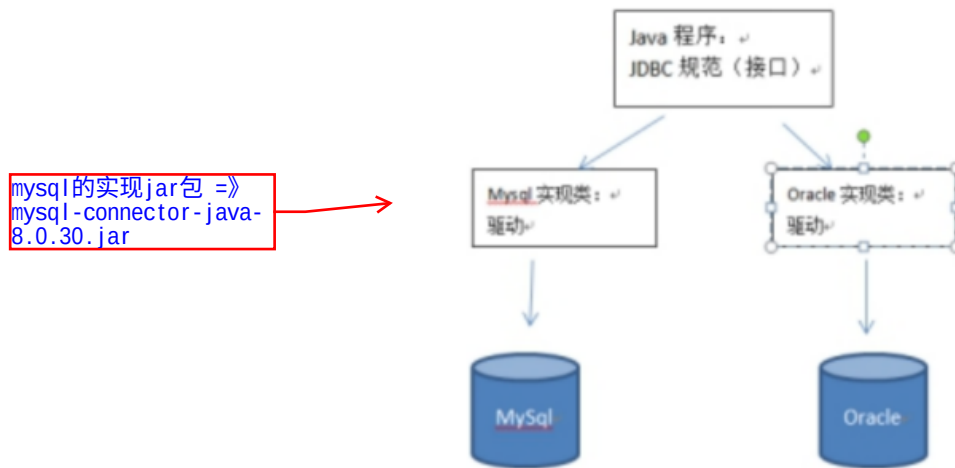
官方只提供了规范，各个数据库厂商，你想要我连接你，你就实现这套规范，这其实就是我们导入的驱动包。

1.1.2 什么是数据库驱动

JDBC需要连接驱动，驱动是两个设备要进行通信，满足一定通信数据格式，数据格式由设备提供商规定，设备提供

商为设备提供驱动软件，通过软件可以与该设备进行通信。

JDBC与数据库驱动的关系:接口与实现的关系。



JDBC规范 (掌握四个核心对象) :

DriverManager:用于注册驱动

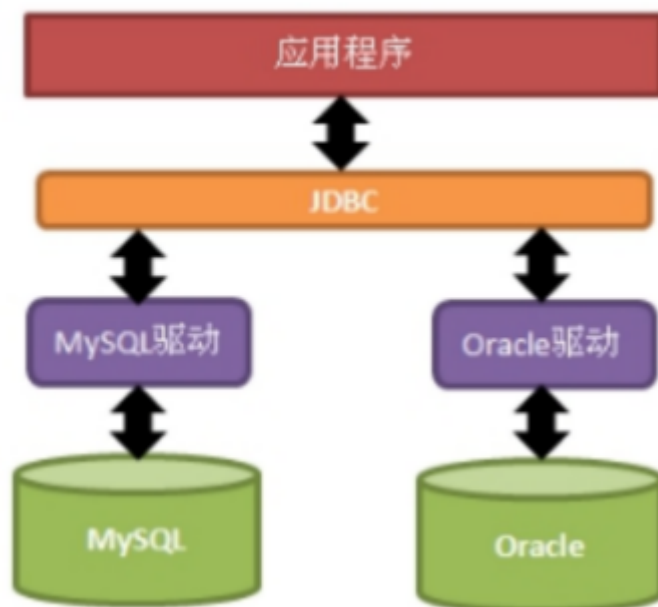
Connection: 表示与数据库创建的连接

Statement: 操作数据库sql语句的对象

ResultSet: 结果集或一张虚拟表

1.2 JDBC原理

Java提供访问数据库规范称为JDBC, 而生产厂商提供规范的实现类称为驱动。



JDBC是接口, 驱动是接口的实现, 没有驱动将无法完成数据库连接, 从而不能操作数据库! 每个数据库厂商都需要

提供自己的驱动, 用来连接自己公司的数据库, 也就是说驱动一般都由数据库生成厂商提供。

JDBC全称为: Java Data Base Connectivity (java数据库连接), 它主要由接口组成。组成JDBC的2个包:

java.sql javax.sql 开发JDBC应用需要以上2个包的支持外, 还需要导入相应JDBC的数据库实现(即数据库驱动)。

1.3 入门程序

rs.next()方法就是往
表格下移一行, 如果
没有下一行返回false

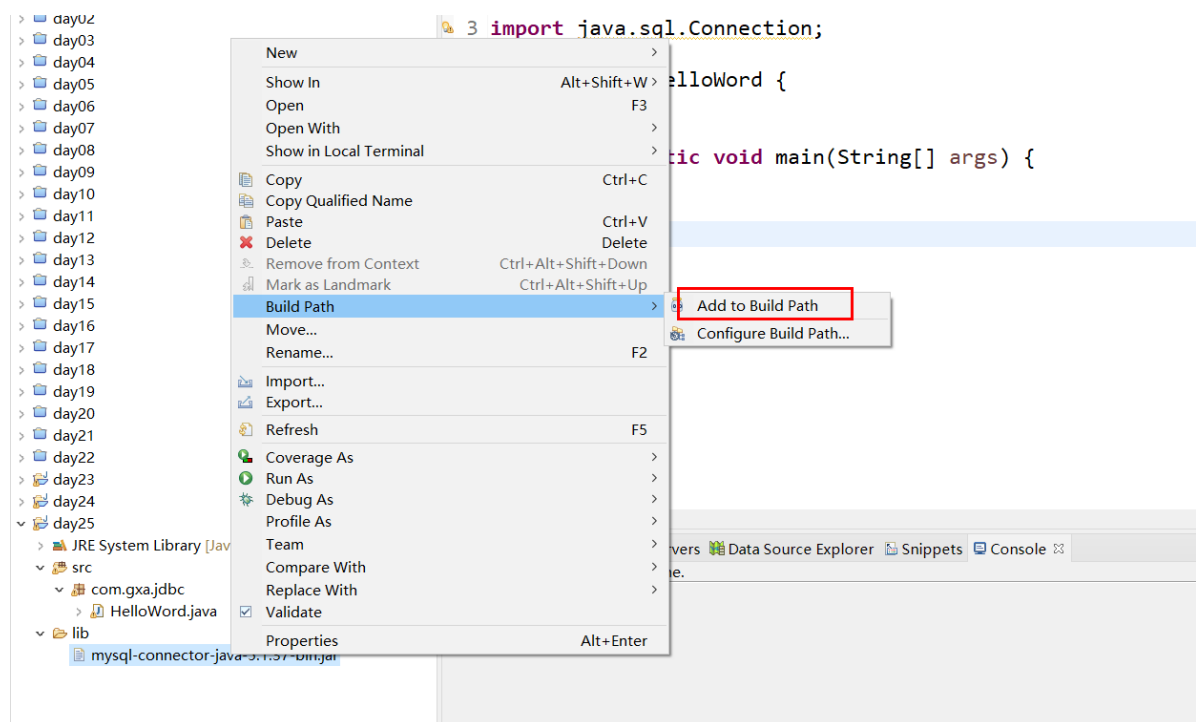
husband

rs →

rs.getObject("id")

id	name	age		
1	奥利给	22		

导包



```

create table user(
    id int primary key auto_increment,
    name varchar(60),
    age int
);

insert into user(name,age) values('奥利给',22);
insert into user(name,age) values('王总',23);
insert into user(name,age) values('给料',22);

```

注册驱动

```

7
8
9 public static void main(String[] args) throws Exception {
10
11     //1. 注册驱动 告诉jdbc我们使用哪一个数据库厂商的驱动
12
13     // 驱动管理器专门注册驱动(需要传递一个驱动对象)
14     DriverManager.registerDriver(null);
15
16
17     //2. 通过驱动管理器获取一个链接
18
19
20     //3. 获取向数据库发送sql的statement对象
21
22
23     //4. 发送sql后获得一个封装了查询结果的ResultSet对象
24
25

```

```

public class Helloworld {

    public static void main(String[] args) throws Exception {

        //1. 注册驱动 告诉jdbc我们使用哪一个数据库厂商的驱动

        //驱动管理器专门注册驱动(需要传递一个驱动对象)
        DriverManager.registerDriver(new com.mysql.jdbc.Driver());

        //2. 通过驱动管理器获取一个链接
        //url: 链接数据库的地址 jdbc:mysql://localhost:3306/数据库的名字
        //user: 用户名
        //password: 密码
        //面向接口编程

        Connection connection = (Connection)
        DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb2", "root",
        "root");
    }
}

```

```

//3.创建向数据库发送sql的statement对象
Statement st = connection.createStatement();

//4.发送sql后获得一个封装了查询结果的ResultSet对象
ResultSet rs = st.executeQuery("select * from user where id=1");

//5.解析ResultSet对象获得结果

if(rs.next()) {

    System.out.println("id:"+rs.getObject("id"));
    System.out.println("name:"+rs.getObject("name"));
    System.out.println("age:"+rs.getObject("age"));

}

//释放资源
rs.close();
st.close();
connection.close();

}

}

```

2.Api详解

2.1 注册驱动

操作数据库步骤

1. 加载驱动
2. 获取链接
3. 准备语句
4. 获取PreparedStatement对象
5. 设置参数
6. 执行语句
7. 处理结果
8. 释放资源

DriverManager.registerDriver(new com.mysql.jdbc.Driver());不建议使用

原因有2个:

>导致驱动被注册2次。

>强烈依赖数据库的驱动jar

解决办法:

注册驱动的第二种方式

```
Class.forName("com.mysql.jdbc.Driver");
```

2.2 获取链接

```
static Connection getConnection(String url, String user, String password)
```

试图建立到给定数据库 URL 的连接。

参数说明: url 需要连接数据库的位置 (网址) user用户名 password 密码

例如: `getConnection("jdbc:mysql://localhost:3306/day06", "root", "root");`

URL: SUN公司与数据库厂商之间的一种协议。

`jdbc:mysql://localhost:3306/day06`

协议 子协议 IP : 端口号 数据库

常用数据库URL地址的写法:

Oracle写法: `jdbc:oracle:thin:@localhost:1521:sid`

SqlServer `jdbc:microsoft:sqlserver://localhost:1433; DatabaseName=sid`

MySQL `jdbc:mysql://localhost:3306/sid`

MySQL的url地址的简写形式: `jdbc:mysql:///sid`

常用属性: `useUnicode=true&characterEncoding=UTF-8`

接口的实现在数据库驱动中。所有与数据库交互都是基于连接对象的。

`Statement createStatement();` //创建操作sql语句的对象

将Statement换成
PreparedStatement比较好

2.3 API详解: java.sql.Statement接口: 操作sql语句, 并返回相应结果

```
String sql = "某SQL语句";
```

获取Statement语句执行平台: `Statement stmt = con.createStatement();`

常用方法:

`int executeUpdate(String sql);` --执行insert update delete语句.

`ResultSet executeQuery(String sql);` --执行select语句.

`boolean execute(String sql);` --仅当执行select并且有结果时才返回true, 执行其他的语句返回false.

常规

高级

数据库

SSL

SSH

HTTP



Navicat



数据库

连接名:

localhost

主机:

localhost

端口:

3306

用户名:

root

密码:

●●●●

☒ 保存密码

测试连接

确定

取消

常规 SQL 预览

数据库名: jdbc

字符集: utf8

排序规则: utf8_general_ci

确定

取消

对象 * 无标题 @jdbc (localhost) - 表

保存 添加字段 插入字段 删除字段 主键 上移 下移

字段 索引 外键 触发器 选项 注释 SQL 预览

名	类型	长度	小数点	不是 null	键	注释
id	int			☒	1	
name	varchar	60		☐		
age	int			☐		

默认:

- ☒ 自动递增
- ☐ 无符号
- ☐ 填充零

//1. 注册驱动

```
Class.forName("com.mysql.jdbc.Driver");
```



```
//2. 获取链接
Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbc","root","root");

//3. 获取代表向数据库发送sql的statement对象
Statement st = conn.createStatement();

String sql="insert into student(name,age) values('奥利给','20')";

//4. 发送sql
//返回影响的行数 大于0代表执行成功
int result = st.executeUpdate(sql);

if(result>0) {
    System.out.println("插入成功");
}

//5. 释放资源
st.close();
conn.close();
```

2.4 API详解：处理结果集（注：执行insert、update、delete无需处理）

ResultSet实际上就是一张二维的表格，我们可以调用其boolean next()方法指向某行记录，当第一次调用next()方法

时，便指向第一行记录的位置，这时就可以使用ResultSet提供的getXXX(int col)方法(与索引从0开始不同个，列从1

开始)来获取指定列的数据：

rs.next();//指向下一行

rs.getObject(1);//获取第一行第一列的数据

ResultSet实际上就是一张二维的表格，我们可以调用其boolean next()方法指向某行记录，当第一次调用next()方法时，便指向第一行记录的位置

boolean next() 向下移动箭头 移动返回false

ResultSet

默认指定表头

id	name	age
1	奥利给	20
2	效力给	21
3	给给给	22

rs.getObject("列名")

rs.getObject(序号);

```
while(rs.next()) {  
    System.out.print(rs.getObject("id"));|  
    System.out.print(rs.getObject("name"));  
    System.out.print(rs.getObject("age"));  
}
```

```
public class QueryDemo {  
  
    public static void main(String[] args) throws Exception {  
  
        //1.注册驱动  
        Class.forName("com.mysql.jdbc.Driver");  
  
        //2.获取链接  
        Connection conn =  
        DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbc","root","root");  
  
        //3.获取代表向数据库发送sql的statement对象  
        Statement st = conn.createStatement();  
  
        String sql="select id,name,age from student";  
  
        //4.发送sql  
        ResultSet rs = st.executeQuery(sql);  
  
        while(rs.next()) {  
  
            //也可以写每一列的序号 但是不推荐使用序号  
            System.out.print(rs.getObject(1));  
            System.out.print(rs.getObject("name"));  
            System.out.print(rs.getObject("age"));  
  
            System.out.println();  
  
        }  
  
        //5.释放资源  
        rs.close();  
    }  
}
```

```

        st.close();
        conn.close();

    }

}

```

思考一下：我们出来的数据，用一个Student对象封装起来

```

public static void main(String[] args) throws Exception {

    //1.注册驱动
    Class.forName("com.mysql.jdbc.Driver");

    //2.获取链接
    Connection conn =
    DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbc","root","root");

    //3.获取代表向数据库发送sql的statement对象
    Statement st = conn.createStatement();

    String sql="select id,name,age from student";

    //4.发送sql
    ResultSet rs = st.executeQuery(sql);

    //定义集合封装Student数据
    List<Student> list=new ArrayList<Student>();

    while(rs.next()) {

        //查询出来的结果封装到对象中
        Student s=new Student();

        //强转好难受
        // s.setId((Integer)rs.getObject("id"));
        // s.setAge((Integer)rs.getObject("age"));
        // s.setName((String)rs.getString("name"));

        s.setId(rs.getInt("id"));
        s.setAge(rs.getInt("age"));
        s.setName(rs.getString("name"));

        list.add(s);
    }

    System.out.println(list);
}

```

```

//5.释放资源
rs.close();
st.close();
conn.close();

}

```

常用方法:

n Object getObject(int index) / Object getObject(String name) 获得任意对象

n String getString(int index)/ String getString(String name) 获得字符串

n int getInt(int index)/int getInt(String name) 获得整形

n double getDouble(int index)/ double getDouble(String name) 获得双精度浮点型

SQL类型	Jdbc对应方法	返回类型
BIT(1) bit(10)	getBoolean getBytes()	Boolean byte[]
TINYINT	getByte()	Byte
SMALLINT	getShort()	Short
Int	getInt()	Int
BIGINT	getLong()	Long
CHAR, VARCHAR, LONGVARCHAR	getString()	String
Text(clob) Blob	getClob getBlob()	Clob Blob
DATE	getDate()	java.sql.Date
TIME	getTime()	java.sql.Time
TIMESTAMP	getTimestamp()	java.sql.Timestamp

2.5 释放资源

与IO流一样，使用后的东西都需要关闭！关闭的顺序是先得到的后关闭，后得到的先关闭。

```
rs.close();
```

```
stmt.close();
```

```
con.close();
```

2.6 关闭异常处理

```
public static void main(String[] args) throws Exception {
```

```
//1.注册驱动
Class.forName("com.mysql.jdbc.Driver");

//2.获取链接
Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/jdbc","root","root");
Statement st=null;
ResultSet rs=null;

try {

//3.获取代表向数据库发送sql的statement对象
st = conn.createStatement();

String sql="select id,name,age from student";

//4.发送sql
rs = st.executeQuery(sql);

//定义集合封装Student数据
List<Student> list=new ArrayList<Student>();

while(rs.next()) {

//查询出来的结果封装到对象中
Student s=new Student();

s.setId(rs.getInt("id"));
s.setAge(rs.getInt("age"));
s.setName(rs.getString("name"));

list.add(s);
}

System.out.println(list);

}finally {

//5.释放资源

//关闭资源之前一定要判断
if(rs!=null) {

try {
rs.close();
}catch (Exception e) {
e.printStackTrace();
}

//让jvm回收没有被关闭的rs对象
rs=null;
}
}
```

```

        if(st!=null) {

            try {
                st.close();

            }catch (Exception e) {
                e.printStackTrace();
            }
            st=null;
        }

        if(conn!=null) {

            try {
                conn.close();
            }catch (Exception e) {
                e.printStackTrace();
            }

            conn=null;
        }

    }

}
}

```

2.7 properties

java.util

类 Properties

java.lang.Object

└ java.util.Dictionary<K, V>

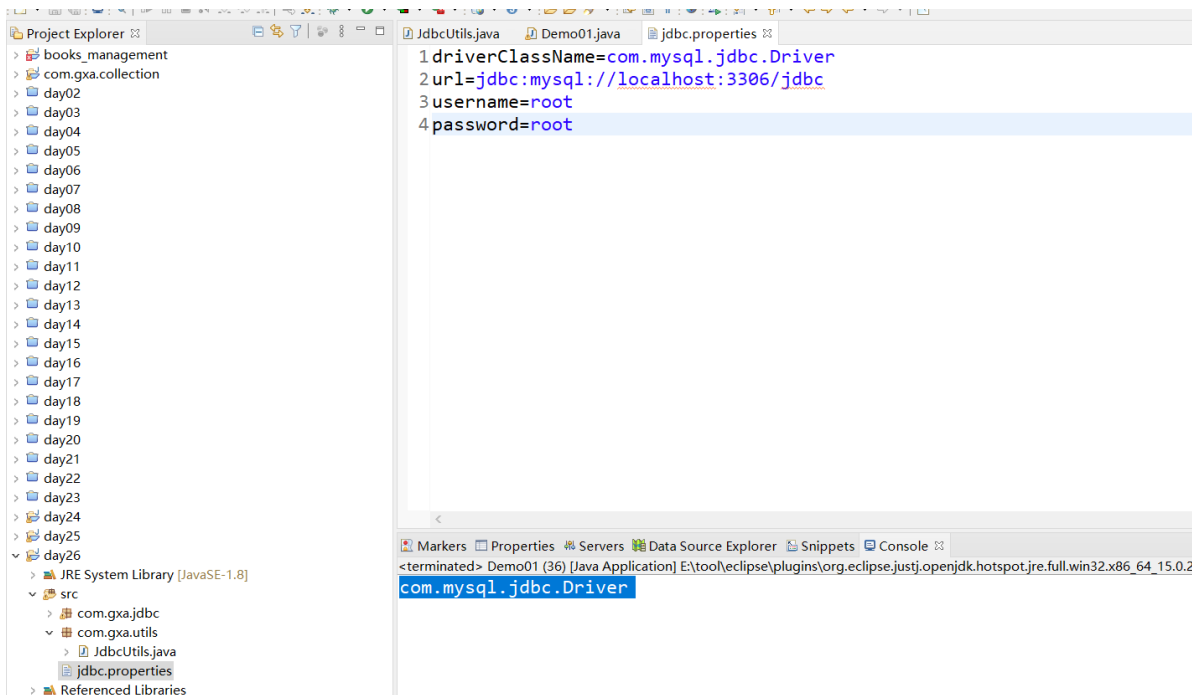
└ java.util.Hashtable<Object, Object>

└ java.util.Properties 

所有已实现的接口：

Serializable, Cloneable, Map<Object, Object>

Properties是一种特殊的集合，唯一一种和io结合起来的集合



```
public class Demo01 {  
  
    public static void main(String[] args) throws Exception {  
  
        //1.创建Properties集合  
        Properties properties=new Properties();  
  
        //void load(InputStream inStream)  
        //从输入流中读取属性列表（键和元素对）。  
  
        //2.创建流加载properties文件
```

```

FileInputStream in = new
FileInputStream("E:\\code\\code\\J257\\day26\\src\\jdbc.properties");

//3.加载properties文件资源
properties.load(in);

//4.String getProperty(String key)
//用指定的键在此属性列表中搜索属性。
String driverClassName = properties.getProperty("driverClassName");

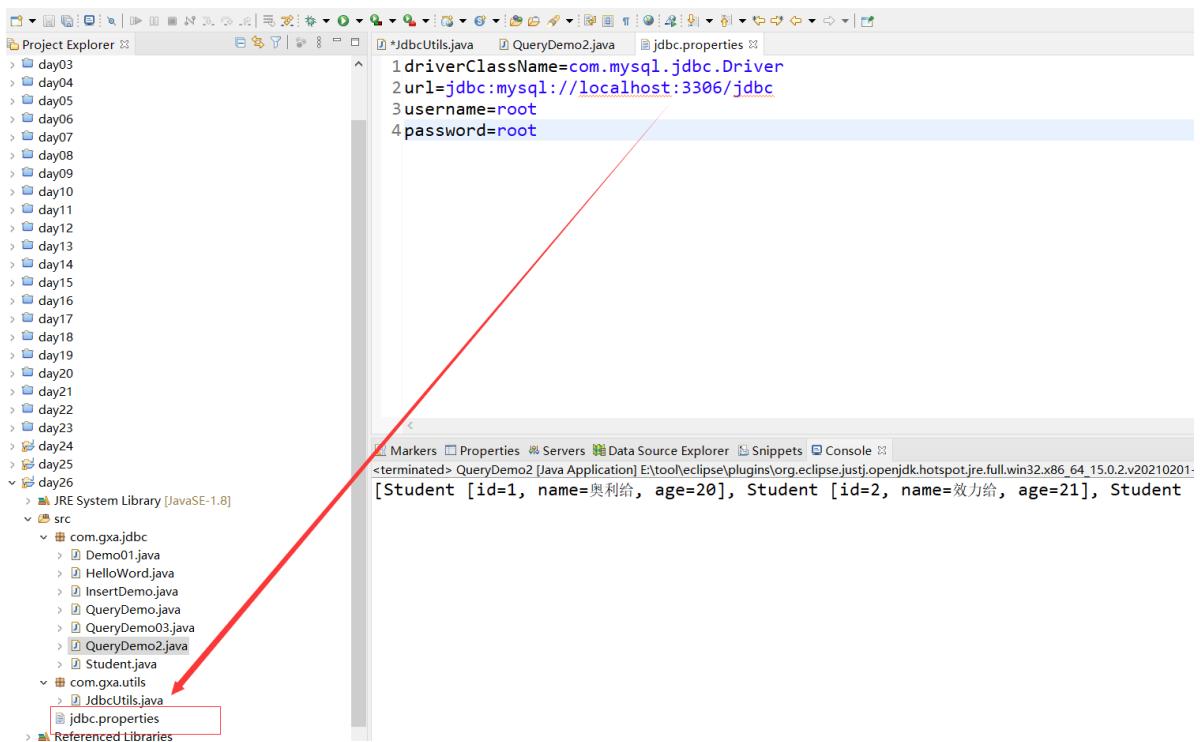
System.out.println(driverClassName);

}

}

```

2.8 JdbcUtils



```

/**
 *
 * 操作Jdbc的工具类
 *
 * @author zxd
 *
 * 2021年5月12日
 */
public class JdbcUtils {

```



```

private static Properties config=new Properties();

//静态代码块 在加载类的时候执行
static {

    try {

        //1.加载配置文件
        //使用的绝对路径 后面替换成相对路径
        FileInputStream in=new
FileInputStream("E:\\code\\code\\J257\\day26\\src\\jdbc.properties");

        //2.把properties文件的资源读取到properties集合中
        config.load(in);

        //3.注册驱动
        Class.forName(config.getProperty("driverClassName"));

    } catch (Exception e) {
        e.printStackTrace();

        //把编译时异常转换成运行异常
        throw new RuntimeException("加载驱动失败!");
    }

}

/**
 * 获取链接
 * @return
 */
public static Connection getConnection() {
    try {

        Connection connection =
DriverManager.getConnection(config.getProperty("url"),
            config.getProperty("username"),
            config.getProperty("password"));

        return connection;
    } catch (SQLException e) {
        e.printStackTrace();
        throw new RuntimeException("获取链接失败!");
    }
}

/**
 * 释放资源
 * @param conn
 * @param st
 * @param rs
 */
public static void release(Connection conn, Statement st, ResultSet rs) {
    //关闭资源之前一定要判断

```

```

        if(rs!=null) {

            try {
                rs.close();
            }catch (Exception e) {
                e.printStackTrace();
            }
            //让jvm回收没有被关闭的rs对象
            rs=null;
        }

        if(st!=null) {

            try {
                st.close();

            }catch (Exception e) {
                e.printStackTrace();
            }
            st=null;
        }

        if(conn!=null) {

            try {
                conn.close();
            }catch (Exception e) {
                e.printStackTrace();
            }

            conn=null;
        }
    }
}

```

```

public class QueryDemo2 {

    public static void main(String[] args) throws Exception {

        //2.获取链接
        Connection conn = JdbcUtils.getConnection();
        Statement st=null;
        ResultSet rs=null;

        try {

            //3.获取代表向数据库发送sql的statement对象

```

```

        st = conn.createStatement();

        String sql="select id,name,age from student";

        //4.发送sql
        rs = st.executeQuery(sql);

        //定义集合封装Student数据
        List<Student> list=new ArrayList<Student>();

        while(rs.next()) {

            //查询出来的结果封装到对象中
            Student s=new Student();

            s.setId(rs.getInt("id"));
            s.setAge(rs.getInt("age"));
            s.setName(rs.getString("name"));

            list.add(s);
        }

        System.out.println(list);

    }finally {

        jdbcUtils.release(conn, st, rs);

    }

}

}

```

2.9 PreparedStatement

PreparedStatement 与 Statement 区别
Statement只能使用字符串拼接的形式向sql传参
;会出现sql注入问题

1.PreparedStatement提前编译sql 提高性能

2.避免sql注入

3.设置值非常方便

```

public class UserDaoImpl implements UserDao{

```

- execute() : 增删改查的sql语句 返回值为boolean -> 是否有结果集

- 可以通过使用
getResultSet()获取结果集

- executeQuery() : 查询 返回值 ResultSet 结果集

- executeUpdate() : 增删改 返回值int -> 影响的数据库中表的行数

```

@Override
public void addUser(User user) {

    //1.获取链接
    Connection conn=JdbcUtils.getConnection();
    PreparedStatement st=null;
    ResultSet rs=null;

    //1.PreparedStatement提前编译sql 提高性能
    //2.避免sql注入
    //3.设置值非常方便

    try {

        //2.准备sql
        //?占位符 代表值，往?设置值
        String sql="insert into user(username,password) values(?,?)";

        //3.获取代表向数据库发送sql的PreparedStatement对象
        st=conn.prepareStatement(sql);

        //设置值
        st.setString(1, user.getUsername());
        st.setString(2, user.getPassword());

        //4.发送sql
        int result = st.executeUpdate();

        //5.判断结果
        if(result>0) {
            System.out.println("插入成功!");
        }

    }catch (Exception e) {
        e.printStackTrace();
        throw new RuntimeException(e);
    }finally {
        JdbcUtils.release(conn, st, rs);
    }

}

```

```

@Override
public void updateUser(User user) {

    //1.获取链接
    Connection conn=JdbcUtils.getConnection();
    PreparedStatement st=null;
    ResultSet rs=null;
    try {

```

```
id=?";

//2.准备sql
String sql="update user set username=?,password=? where
```

```


//3.获取代表向数据库发送sql的PreparedStatement对象
st=conn.prepareStatement(sql);
st.setString(1, user.getUsername());
st.setString(2, user.getPassword());
st.setInt(3, user.getId());
```

```


//4.发送sql
int result = st.executeUpdate();
```

```


//5.判断结果
if(result>0) {
    System.out.println("修改成功!");
}
```

```

}catch (Exception e) {
    e.printStackTrace();
    throw new RuntimeException(e);
}finally {
    JdbcUtils.release(conn, st, rs);
}
```

```
}
```

```
@Override
```

```
public void deleteUserById(Integer id) {
```

```


//1.获取链接
Connection conn=JdbcUtils.getConnection();
PreparedStatement st=null;
ResultSet rs=null;
try {
```

```


//2.准备sql
String sql="delete from user where id=?";
```

```


//3.获取代表向数据库发送sql的PreparedStatement对象
st=conn.prepareStatement(sql);
```

```
st.setInt(1, id);
```

```


//4.发送sql
int result = st.executeUpdate();
```

```


//5.判断结果
if(result>0) {
    System.out.println("删除成功!");
}
```

```

    }

    }catch (Exception e) {
        e.printStackTrace();
        throw new RuntimeException(e);
    }finally {
        JdbcUtils.release(conn, st, rs);
    }

}

@Override
public User findUserById(Integer id) {

    Connection conn=JdbcUtils.getConnection();
    PreparedStatement st=null;
    ResultSet rs=null;

    try {

        //1.准备sql
        String sql="select id,username,password from user where id=?";

        //2.获取代表向数据库发送sql的PreparedStatement对象
        st=conn.prepareStatement(sql);

        st.setInt(1, id);

        //3.执行sql, 返回一个封装了结果的ResultSet对象
        rs=st.executeQuery();

        //4.解析ResultSet

        if(rs.next()) {

            User user=new User();
            user.setId(rs.getInt("id"));
            user.setUsername(rs.getString("username"));
            user.setPassword(rs.getString("password"));

            return user;
        }

        return null;

    }catch (Exception e) {
        e.printStackTrace();
        throw new RuntimeException(e);
    }finally {
        JdbcUtils.release(conn, st, rs);
    }

}

```

```

@Override
public User findUserByUsernameAndPassword(String username, String password)
{

    Connection conn=JdbcUtils.getConnection();
    PreparedStatement st=null;
    ResultSet rs=null;

    try {

        //1.准备sql
        String sql="select id,username,password from user where username=?
and password=?";

        //2.获取代表向数据库发送sql的PreparedStatement对象
        st=conn.prepareStatement(sql);

        st.setString(1, username);
        st.setString(2, password);

        //3.执行sql，返回一个封装了结果的ResultSet对象
        rs=st.executeQuery();

        //4.解析ResultSet

        if(rs.next()) {

            User user=new User();
            user.setId(rs.getInt("id"));
            user.setUsername(rs.getString("username"));
            user.setPassword(rs.getString("password"));

            return user;
        }

        return null;

    }catch (Exception e) {
        e.printStackTrace();
        throw new RuntimeException(e);
    }finally {
        JdbcUtils.release(conn, st, rs);
    }
}

@Override
public List<User> findUserAll() {

    Connection conn=JdbcUtils.getConnection();
    Statement st=null;
    ResultSet rs=null;

    try {

```

```

//1.准备sql
String sql="select id,username,password from user";

//2.获取代表向数据库发送sql的statement对象
st=conn.createStatement();

//3.执行sql，返回一个封装了结果的ResultSet对象
rs=st.executeQuery(sql);

//4.解析ResultSet

List<User> list=new ArrayList<User>();

while(rs.next()) {

    User user=new User();
    user.setId(rs.getInt("id"));
    user.setUsername(rs.getString("username"));
    user.setPassword(rs.getString("password"));
    list.add(user);
}

return list;

}catch (Exception e) {
    e.printStackTrace();
    throw new RuntimeException(e);
}finally {
    JdbcUtils.release(conn, st, rs);
}
}
}

```

2.10 结果集元数据api

```

@Test
public void test05() throws Exception{
    List<User> users = new ArrayList<>();
    Connection connection = DBConnection.getConnection();

    String sql = "SELECT * FROM t_user";

    PreparedStatement ps = connection.prepareStatement(sql);

    ResultSet rs = ps.executeQuery();
    //结果集的元数据 （描述结果集的数据）
    ResultSetMetaData rsMetaData = rs.getMetaData();
    //      System.out.println("一共有" + rsMetaData.getColumnCount() + "列");
    int columnCount = rsMetaData.getColumnCount();
    //      for(int i=1;i<=columnCount;i++){

```

有些像元注解（描述注解的注解）


```
//      String columnName = rsMetaData.getColumnName(i);
//      String columnTypeName = rsMetaData.getColumnTypeName(i);
//
//      System.out.println(columnName + "-----" + columnTypeName);
//      while(rs.next()) {
//          Object obj = rs.getObject(columnName);
//          System.out.println(obj);
//      }
//
//
//
//      }
```

能获取到表中字段名和
数据类型

```
while(rs.next()) {
    for(int i=1;i<=columnCount;i++) {
        String columnName = rsMetaData.getColumnName(i);
        Object obj = rs.getObject(columnName);
        System.out.println(obj);
    }

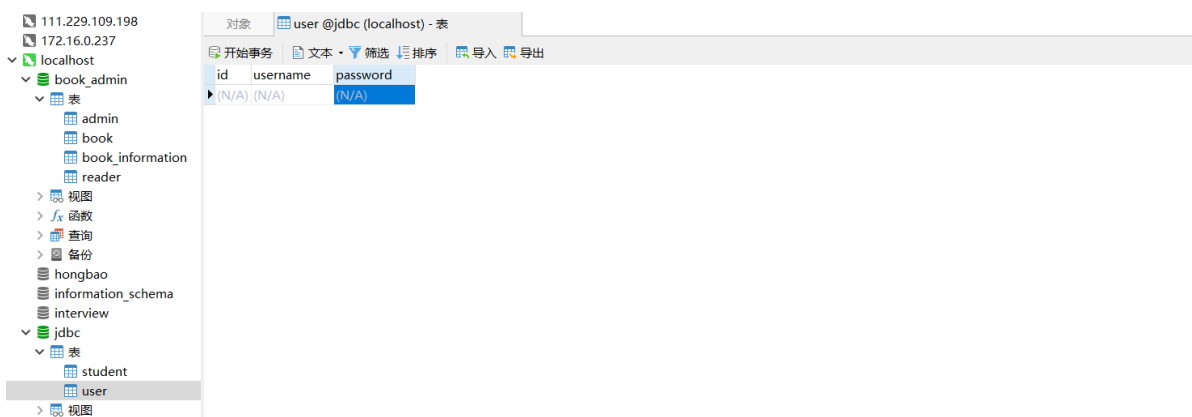
    System.out.println("=====");
}

DBConnection.close(rs,ps,connection);

}
```

3.crud

3.1 编写user类



```
public class User {

    private Integer id;
```

```
private String username;

private String password;


public User() {
}


public User(Integer id, String username, String password) {
    this.id = id;
    this.username = username;
    this.password = password;
}


public Integer getId() {
    return id;
}


public void setId(Integer id) {
    this.id = id;
}


public String getUsername() {
    return username;
}


public void setUsername(String username) {
    this.username = username;
}


public String getPassword() {
    return password;
}


public void setPassword(String password) {
    this.password = password;
}
```

```

@Override
public String toString() {
    return "User [id=" + id + ", username=" + username + ", password=" +
password + "]\n";
}

}

```

3.2 编写接口

```

public interface UserDao {

    //添加学生
    public void addUser(User user);

    //修改学生
    public void updateUser(User user);

    //根据id删除一个学生
    public void deleteUserById(Integer id);

    //根据id查询一个用户
    public User findUserById(Integer id);

    //根据用户名和密码查询一个用户
    public User findUserByUsernameAndPassword(String username,String password);

    //查询所有
    public List<User> findUserAll();

}

```

3.3 编写实现类

```

public class UserDaoImpl implements UserDao{

    @Override
    public void addUser(User user) {

        //1. 获取链接
        Connection conn=JdbcUtils.getConnection();
        Statement st=null;
        ResultSet rs=null;
        try {

```

```

        //2.准备sql
        String sql="insert into user(username,password)
values('"+user.getUsername()+"','"+user.getPassword()+"')";

        //3.获取代表向数据库发送sql的statement对象
        st=conn.createStatement();

        //4.发送sql
        int result = st.executeUpdate(sql);

        //5.判断结果
        if(result>0) {
            System.out.println("插入成功!");
        }

    }catch (Exception e) {
        e.printStackTrace();
        throw new RuntimeException(e);
    }finally {
        JdbcUtils.release(conn, st, rs);
    }

}

@Override
public void updateUser(User user) {

    //1.获取链接
    Connection conn=JdbcUtils.getConnection();
    Statement st=null;
    ResultSet rs=null;
    try {

        //2.准备sql
        String sql="update user set
username='"+user.getUsername()+"',password='"+user.getPassword()+"' where
id='"+user.getId()+"'";

        //3.获取代表向数据库发送sql的statement对象
        st=conn.createStatement();

        //4.发送sql
        int result = st.executeUpdate(sql);

        //5.判断结果
        if(result>0) {
            System.out.println("修改成功!");
        }
    }

```

```

        }catch (Exception e) {
            e.printStackTrace();
            throw new RuntimeException(e);
        }finally {
            JdbcUtils.release(conn, st, rs);
        }
    }

    @Override
    public void deleteUserById(Integer id) {

        //1. 获取链接
        Connection conn=JdbcUtils.getConnection();
        Statement st=null;
        ResultSet rs=null;
        try {

            //2. 准备sql
            String sql="delete from user where id="+id;

            //3. 获取代表向数据库发送sql的statement对象
            st=conn.createStatement();

            //4. 发送sql
            int result = st.executeUpdate(sql);

            //5. 判断结果
            if(result>0) {
                System.out.println("删除成功!");
            }

        }catch (Exception e) {
            e.printStackTrace();
            throw new RuntimeException(e);
        }finally {
            JdbcUtils.release(conn, st, rs);
        }
    }

    @Override
    public User findUserById(Integer id) {

        Connection conn=JdbcUtils.getConnection();
        Statement st=null;
        ResultSet rs=null;

        try {

            //1. 准备sql
            String sql="select id,username,password from user where id="+id;

```

```

//2. 获取代表向数据库发送sql的statement对象
st=conn.createStatement();

//3. 执行sql, 返回一个封装了结果的ResultSet对象
rs=st.executeQuery(sql);

//4. 解析ResultSet

if(rs.next()) {

    User user=new User();
    user.setId(rs.getInt("id"));
    user.setUsername(rs.getString("username"));
    user.setPassword(rs.getString("password"));

    return user;
}

return null;

}catch (Exception e) {
    e.printStackTrace();
    throw new RuntimeException(e);
}finally {
    jdbcUtils.release(conn, st, rs);
}

}

@Override
public User findUserByUsernameAndPassword(String username, String password)
{

    Connection conn=jdbcUtils.getConnection();
    Statement st=null;
    ResultSet rs=null;

    try {

        //1. 准备sql
        String sql="select id,username,password from user where
username='"+username+"' and password='"+password+"'";

        //2. 获取代表向数据库发送sql的statement对象
        st=conn.createStatement();

        //3. 执行sql, 返回一个封装了结果的ResultSet对象
        rs=st.executeQuery(sql);

        //4. 解析ResultSet

        if(rs.next()) {

```

```

        User user=new User();
        user.setId(rs.getInt("id"));
        user.setUsername(rs.getString("username"));
        user.setPassword(rs.getString("password"));

        return user;
    }

    return null;

} catch (Exception e) {
    e.printStackTrace();
    throw new RuntimeException(e);
} finally {
    JdbcUtils.release(conn, st, rs);
}
}

@Override
public List<User> findUserAll() {

    Connection conn=JdbcUtils.getConnection();
    Statement st=null;
    ResultSet rs=null;

    try {

        //1.准备sql
        String sql="select id,username,password from user";

        //2.获取代表向数据库发送sql的statement对象
        st=conn.createStatement();

        //3.执行sql，返回一个封装了结果的ResultSet对象
        rs=st.executeQuery(sql);

        //4.解析ResultSet

        List<User> list=new ArrayList<User>();

        while(rs.next()) {

            User user=new User();
            user.setId(rs.getInt("id"));
            user.setUsername(rs.getString("username"));
            user.setPassword(rs.getString("password"));
            list.add(user);
        }

        return list;

    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

        throw new RuntimeException(e);
    }finally {
        jdbcUtils.release(conn, st, rs);
    }
}
}

```

3.4 编写实现类

```

public class UserDaoTest {

    public static void main(String[] args) {

        //1.测试添加
        UserDao userDao=new UserDaoImpl();

        User user=new User(null, "alg", "123456");

        userDao.addUser(user);

        //2.测试修改
        // UserDao userDao=new UserDaoImpl();
        //
        // User user=new User(1, "alg2", "1234562");
        //
        // userDao.updateUser(user);

        //3.测试删除
        // UserDao userDao=new UserDaoImpl();
        // userDao.deleteUserById(1);

        //4.测试查询

        // UserDao userDao=new UserDaoImpl();
        //// User user = userDao.findUserById(1);
        //// System.out.println(user);
        //
        // User user = userDao.findUserByUsernameAndPassword("alg2", "1234562");
        //// System.out.println(user);
        //
        //
        // List<User> list = userDao.findUserAll();
        // System.out.println(list);

    }

}

```


3.5 作业

今天上课的所有代码

设置一张部门表 Department

id

name 名字

number 部门编号

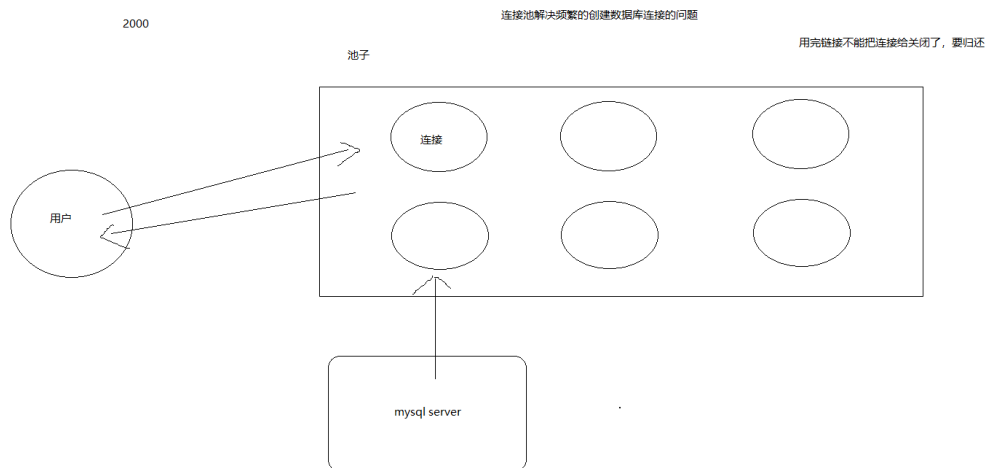
count 人数

crud代码

4.数据库连接池

避免重复打开 或者 关闭数据库，从而导致性能低下

4.1 什么是数据库连接池



4.2 设计自己的数据库连接池

DataSource: 为大家提供了统一的获取连接的方式。

数据源类

要编写连接池，需要实现一个接口 DataSource

设计模式： 编写代码总结的设计的经验

设计模式是一种思想

设计模式很难，不是在于每一种是什么意思，而在于什么地方去用

```
public class MyDataSource implements DataSource{

    //连接池
    private static LinkedList<Connection> conns=new LinkedList<Connection>();

    //Properties对象读取Properties文件
    private static Properties config=new Properties();

    //在类加载的时候就提前创建连接放到连接池中(LinkedList)
    static {

        try {

            //1.加载配置文件
            FileInputStream in=new
            FileInputStream("E:\\code\\code\\j257\\day28\\src\\jdbc.properties");

            //2.把properties文件的资源读取到properties集合中
            config.load(in);

            //3.注册驱动
            Class.forName(config.getProperty("driverClassName"));

            //4.获取链接放入到池子中
            for(int i=0;i<5;i++) {

                //5.创建链接
                Connection connection =
                DriverManager.getConnection(config.getProperty("url"),
                    config.getProperty("username"),
                    config.getProperty("password"));

                //6.往池子中添加链接
                conns.add(connection);
            }

        }catch (Exception e) {
            e.printStackTrace();
            //把编译时异常转换成运行异常
            throw new RuntimeException("加载驱动失败!");
        }

    }

    /**
```

```

    * 获取连接
    */
@Override
public Connection getConnection() throws SQLException {

    //获取mysql的连接
    Connection connection = conns.removeFirst();

    //用我们自己编写的一个包装类把mysql的Connection给包装了
    return new MyConnection(connection);
}

//1.重写不现实的，因为你不知道用户使用什么数据库的驱动，不同的驱动实现不一样
//2.包装设计模式
//3.动态代理

//1.编写一个类实现与被增强对象相同的接口
//2.定义一个变量接收被增强对象
//3.编写一个构造器接收被增强对象
//4.覆盖想覆盖的方法
//5.对于不想覆盖的方法直接调用被增强对象的方法来实现

//1.编写一个类实现与被增强对象相同的接口
class MyConnection implements Connection{

    //2.定义一个变量接收被增强对象
    private Connection conn;

    //3.编写一个构造器接收被增强对象
    public MyConnection(Connection conn) {
        this.conn=conn;
    }

    //4.覆盖想覆盖的方法
    @Override
    public void close() throws SQLException {

        //把连接还回去(把连接重新加入集合中)
        conns.add(conn);
    }

    //5.对于不想覆盖的方法直接调用被增强对象的方法来实现
    @Override
    public <T> T unwrap(Class<T> iface) throws SQLException {
        return conn.unwrap(iface);
    }

    @Override
    public boolean isWrapperFor(Class<?> iface) throws SQLException {
        return conn.isWrapperFor(iface);
    }
}

```

```

@Override
public Statement createStatement() throws SQLException {
    return conn.createStatement();
}

@Override
public PreparedStatement prepareStatement(String sql) throws
SQLException {
    // TODO Auto-generated method stub
    return null;
}

@Override
public CallableStatement prepareCall(String sql) throws SQLException {
    // TODO Auto-generated method stub
    return null;
}

@Override
public String nativeSQL(String sql) throws SQLException {
    // TODO Auto-generated method stub
    return null;
}

@Override
public void setAutoCommit(boolean autoCommit) throws SQLException {
    // TODO Auto-generated method stub
}

@Override
public boolean getAutoCommit() throws SQLException {
    // TODO Auto-generated method stub
    return false;
}

@Override
public void commit() throws SQLException {
    // TODO Auto-generated method stub
}

@Override
public void rollback() throws SQLException {
    // TODO Auto-generated method stub
}

@Override
public boolean isClosed() throws SQLException {
    // TODO Auto-generated method stub
    return false;
}

```

```
}

@Override
public DatabaseMetaData getMetaData() throws SQLException {
    // TODO Auto-generated method stub
    return null;
}

@Override
public void setReadOnly(boolean readOnly) throws SQLException {
    // TODO Auto-generated method stub
}

@Override
public boolean isReadOnly() throws SQLException {
    // TODO Auto-generated method stub
    return false;
}

@Override
public void setCatalog(String catalog) throws SQLException {
    // TODO Auto-generated method stub
}

@Override
public String getCatalog() throws SQLException {
    // TODO Auto-generated method stub
    return null;
}

@Override
public void setTransactionIsolation(int level) throws SQLException {
    // TODO Auto-generated method stub
}

@Override
public int getTransactionIsolation() throws SQLException {
    // TODO Auto-generated method stub
    return 0;
}

@Override
public SQLWarning getWarnings() throws SQLException {
    // TODO Auto-generated method stub
    return null;
}

@Override
public void clearWarnings() throws SQLException {
    // TODO Auto-generated method stub
}
}
```

```

@Override
    public Statement createStatement(int resultSetType, int
resultSetConcurrency) throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

@Override
    public PreparedStatement prepareStatement(String sql, int resultSetType,
int resultSetConcurrency)
        throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

@Override
    public CallableStatement prepareCall(String sql, int resultSetType, int
resultSetConcurrency)
        throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

@Override
    public Map<String, Class<?>> getTypeMap() throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

@Override
    public void setTypeMap(Map<String, Class<?>> map) throws SQLException {
        // TODO Auto-generated method stub

    }

@Override
    public void setHoldability(int holdability) throws SQLException {
        // TODO Auto-generated method stub

    }

@Override
    public int getHoldability() throws SQLException {
        // TODO Auto-generated method stub
        return 0;
    }

@Override
    public Savepoint setSavepoint() throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

@Override

```

```

    public Savepoint setSavepoint(String name) throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public void rollback(Savepoint savepoint) throws SQLException {
        // TODO Auto-generated method stub
    }

    @Override
    public void releaseSavepoint(Savepoint savepoint) throws SQLException {
        // TODO Auto-generated method stub
    }

    @Override
    public Statement createStatement(int resultSetType, int
resultSetConcurrency, int resultSetHoldability)
        throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public PreparedStatement prepareStatement(String sql, int resultSetType,
int resultSetConcurrency,
        int resultSetHoldability) throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public CallableStatement prepareCall(String sql, int resultSetType, int
resultSetConcurrency,
        int resultSetHoldability) throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public PreparedStatement prepareStatement(String sql, int
autoGeneratedKeys) throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public PreparedStatement prepareStatement(String sql, int[]
columnIndexes) throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

```

```

@Override
    public PreparedStatement prepareStatement(String sql, String[]
columnNames) throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

@Override
    public Clob createClob() throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

@Override
    public Blob createBlob() throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

@Override
    public NClob createNClob() throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

@Override
    public SQLXML createSQLXML() throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

@Override
    public boolean isValid(int timeout) throws SQLException {
        // TODO Auto-generated method stub
        return false;
    }

@Override
    public void setClientInfo(String name, String value) throws
SQLException {
        // TODO Auto-generated method stub

    }

@Override
    public void setClientInfo(Properties properties) throws
SQLException {
        // TODO Auto-generated method stub

    }

@Override
    public String getClientInfo(String name) throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

```



```

    }

    @Override
    public Properties getClientInfo() throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public Array createArrayOf(String typeName, Object[] elements) throws
SQLException {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public Struct createStruct(String typeName, Object[] attributes) throws
SQLException {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public void setSchema(String schema) throws SQLException {
        // TODO Auto-generated method stub

    }

    @Override
    public String getSchema() throws SQLException {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public void abort(Executor executor) throws SQLException {
        // TODO Auto-generated method stub

    }

    @Override
    public void setNetworkTimeout(Executor executor, int milliseconds)
throws SQLException {
        // TODO Auto-generated method stub

    }

    @Override
    public int getNetworkTimeout() throws SQLException {
        // TODO Auto-generated method stub
        return 0;
    }

}

```

```

@Override
public Logger getParentLogger() throws SQLFeatureNotSupportedException {
    return null;
}

@Override
public <T> T unwrap(Class<T> iface) throws SQLException {
    return null;
}

@Override
public boolean isWrapperFor(Class<?> iface) throws SQLException {
    return false;
}

@Override
public Connection getConnection(String username, String password) throws
SQLException {
    return null;
}

@Override
public PrintWriter getLogWriter() throws SQLException {
    return null;
}

@Override
public void setLogWriter(PrintWriter out) throws SQLException {

}

@Override
public void setLoginTimeout(int seconds) throws SQLException {

}

@Override
public int getLoginTimeout() throws SQLException {
    return 0;
}

}

```

使用连接池

```

public class DataSourceTest {

```

```

public static void main(String[] args) throws SQLException {

    MyDataSource myDataSource=new MyDataSource();

    Connection connection = myDataSource.getConnection();

    System.out.println(connection);

    //习惯了close方法，这个close是数据库的close方法，关闭连接
    connection.close();

    System.out.println(connection);

}

}

```

反射 动态代理 注解 泛型 框架的底层

如果你要写框架，你就要用这些东西

4.3 定时

```

public class TestTimer {

    public static void main(String[] args) {
        Timer timer = new Timer();

        TimerTask timerTask = new TimerTask() {
            @Override
            public void run() {
                System.out.println(Thread.currentThread().getName());
                System.out.println("111111111111111");
            }
        };

        timer.schedule(timerTask,1000,2000);

    }

}

```

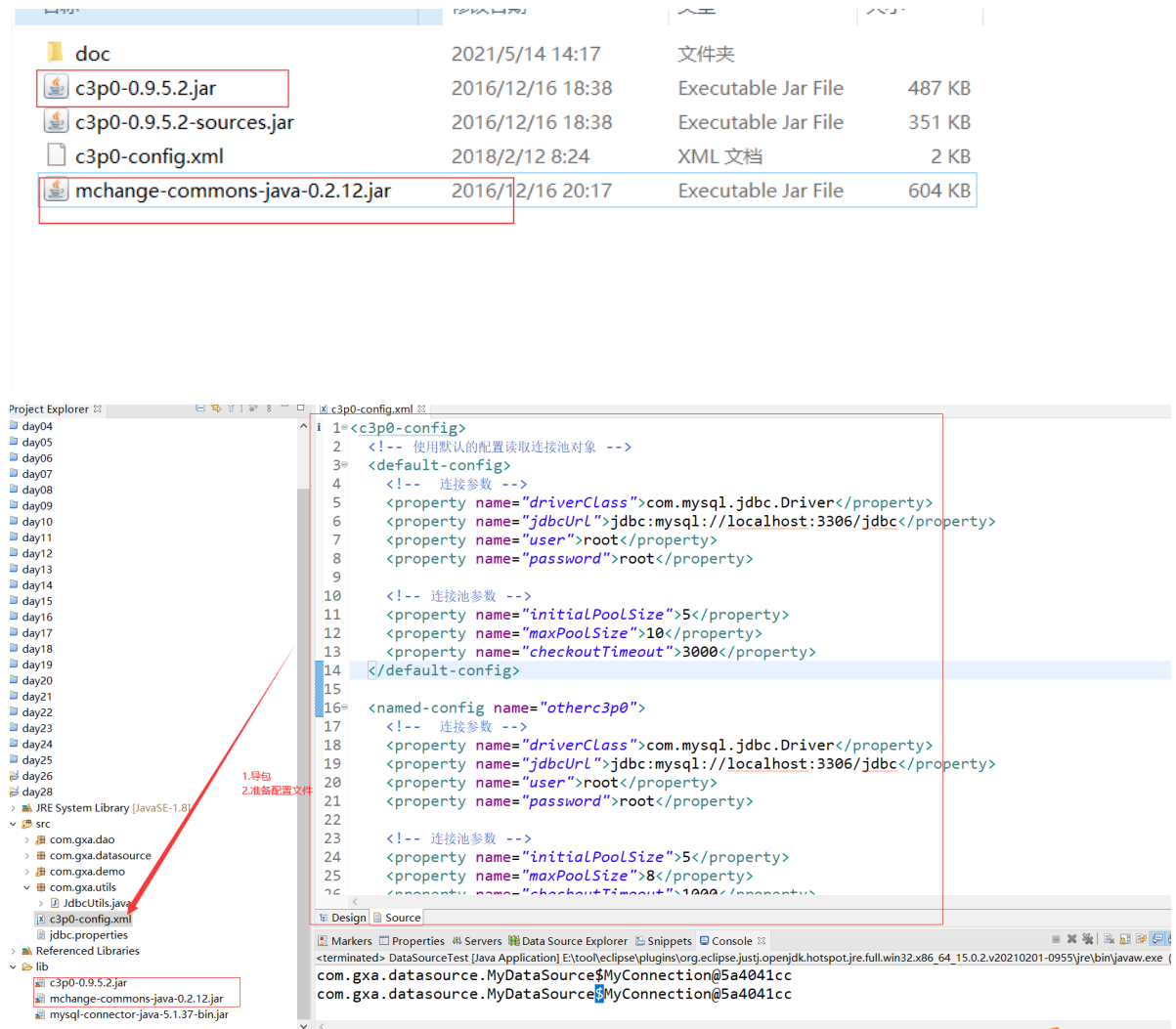
4.4 常见的连接池

dbcp 性能低

c3p0

druid 阿里的

c3p0



使用官方的配置文件

```
//创建c3p0数据库连接池，获取连接
//new ComboPooledDataSource() 自动识别src下面的c3p0-config.xml文件读取默认的连接配置
//DataSource dataSource=new ComboPooledDataSource();

DataSource dataSource=new ComboPooledDataSource("otherc3p0");

Connection conn = dataSource.getConnection();

System.out.println(conn);
```

直接使用set方法设置连接信息

```
ComboPooledDataSource comboPooledDataSource = new
ComboPooledDataSource();

comboPooledDataSource.setDriverClass("com.mysql.jdbc.Driver");
comboPooledDataSource.setJdbcUrl("jdbc:mysql://localhost:3306/jdbc");
comboPooledDataSource.setUser("root");
comboPooledDataSource.setPassword("root");

Connection connection = comboPooledDataSource.getConnection();

System.out.println(connection);
```

使用:

导入依赖:

```
DruidDataSource dataSource=new DruidDataSource();

dataSource.setDriverClassName("com.mysql.jdbc.Driver");
dataSource.setUrl("jdbc:mysql://localhost:3306/jdbc");
dataSource.setUsername("root");
dataSource.setPassword("root");

//初始化连接
dataSource.setInitialSize(5);

//最大活跃连接
dataSource.setMaxActive(10);

Connection connection = dataSource.getConnection();

System.out.println(connection);
```

5、事务的优雅解决

使用ThreadLocal类

← 必须同一线程下放
(set) 和 拿 (get)
才能正确获取数据

其中ThreadLocal的介绍如下:

JDK 1.2的版本中就提供java.lang.ThreadLocal, 为解决多线程程序的并发问题提供了一种新的思路。使用这个工具类可以很简洁地编写出优美的多线程程序。通常用来在多线程中管理共享数据库连接、Session等

ThreadLocal用于保存某个线程共享变量，原因是在Java中，每一个线程对象中都有一个ThreadLocalMap<ThreadLocal, Object>，其key就是一个ThreadLocal，而Object即为该线程的共享变量。而这个map是通过ThreadLocal的set和get方法操作的。对于同一个static ThreadLocal，不同线程只能从中get，set，remove自己的变量，而不会影响其他线程的变量。

- 1、ThreadLocal对象.get: 获取ThreadLocal中当前线程共享变量的值。
- 2、ThreadLocal对象.set: 设置ThreadLocal中当前线程共享变量的值。
- 3、ThreadLocal对象.remove: 移除ThreadLocal中当前线程共享变量的值。



- 1、ThreadLocal对象.get: 获取ThreadLocal中当前线程共享变量的值。
- 2、ThreadLocal对象.set: 设置ThreadLocal中当前线程共享变量的值。
- 3、ThreadLocal对象.remove: 移除ThreadLocal中当前线程共享变量的值。

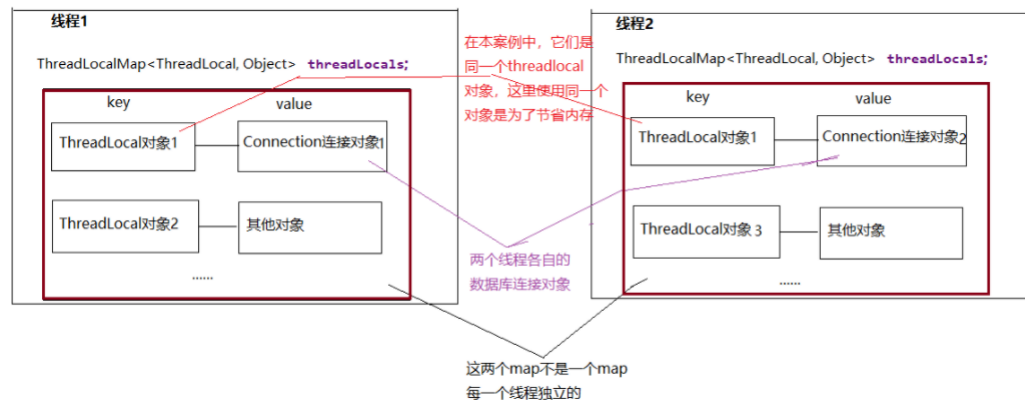
每一个线程对象 调用ThreadLocal对象的get/set/remove方法时，都只会操作“自己线程”内部的ThreadLocalMap对象!!!

```

199 public void set(T value) {
200     Thread t = Thread.currentThread(); 得到当前线程对象自己
201     ThreadLocalMap map = getMap(t); 得到当前线程的ThreadLocalMap对象
202     if (map != null)
203         map.set(this, value); 以ThreadLocal对象为key，set方法的实参为value，把(key,value)存到map中
204     else
205         createMap(t, value);
206 }

```

ThreadLocal用于保存某个线程内部的共享数据，每一个线程对象中都有一个ThreadLocalMap<ThreadLocal, Object>，其key就是一个ThreadLocal，而Object即为该线程整个生命周期中在不同的代码位置需要的同一个共享对象。例如，一个线程对象在同一个事务中多次调用多个dao方法时需要共享使用同一个数据库连接Connection对象。



```

public class JDBCTools {
    //数据库连接池
    private static DataSource ds;

    //静态变量的初始化，可以使用静态代码块
    static{
        try {
            Properties pro = new Properties();

            pro.load(JDBCTools.class.getClassLoader().getResourceAsStream("druid.properties"));

            ds = DruidDataSourceFactory.createDataSource(pro);
        } catch (IOException e) {
            e.printStackTrace();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static ThreadLocal<Connection> threadLocal = new ThreadLocal<>();
    //<Connection>表示 ThreadLocalMap中(key,value)的value是Connection类型的对象

    public static Connection getConnection()throws SQLException {
        Connection connection = threadLocal.get();
    }
}

```

```

        //每一个线程调用这句代码，都会到自己的ThreadLocalMap中，以threadLocal对象为key，
        找到value
        //如果value为空，说明当前线程还未获取过Connection对象，那么就从连接池中拿一个数据库
        连接对象给你
        //并且通过threadLocal的set方法把Connection对象放到当前线程ThreadLocalMap中
        if(connection == null){
            connection = ds.getConnection();
            //通过threadLocal的set方法把Connection对象放到当前线程ThreadLocalMap中
            threadLocal.set(connection);
        }
        return connection;
    }

    public static void freeConnection()throws SQLException{
        Connection connection = threadLocal.get();
        if(connection != null){
            connection.setAutoCommit(true); //还原自动提交模式
            threadLocal.remove(); //从当前线程的ThreadLocalMap中删除这个连接
            connection.close();
        }
    }
}

```

```

/**
 * 创建数据库连接池
 */
public class JDBCUtils {
    // 创建一个ThreadLocal对象，用当前线程作为key
    private static ThreadLocal<Connection> tc = new ThreadLocal<Connection>();
    // 读取的是C3P0-config默认配置创建数据库连接池对象
    private static DataSource ds = new ComboPooledDataSource();

    // 获取数据库连接池对象
    public static DataSource getDataSource() {
        return ds;
    }

    // 从连接池中获取连接
    public static Connection getConnection() throws SQLException {
        Connection conn = tc.get();
        if (conn == null) {
            conn = ds.getConnection();
            // 将conn存放到集合tc中
            tc.set(conn);
            System.out.println("首次创建连接: " + conn);
        }
        return conn;
    }

    // 开启事务
    public static void startTransaction() {
        try {
            // 获取连接
            Connection conn = getConnection();
            // 开启事务

```

```

        /*
        * setAutoCommit总的来说就是保持数据的完整性，一个系统的更新操作可能要涉及多张
        表，需多个SQL语句进行操作
        * 循环里连续的进行插入操作，如果你在开始时设置了：conn.setAutoCommit(false)；
        * 最后才进行conn.commit()，这样你即使插入的时候报错，修改的内容也不会提交到数据
        库，

        */
        conn.setAutoCommit(false);
    } catch (Exception e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}

public static void commit() {
    try {
        Connection conn = tc.get();
        if (conn != null) {
            conn.commit();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public static void rollback() {
    try {
        // 从集合tc中得到一个连接
        Connection conn = tc.get();
        if (conn != null) {
            // 该方法用于取消在当前事务中进行的更改，并释放当前Connection对象持有的所有
            数据库锁。此方法只有在手动事务模式下才可用
            conn.rollback();
        }
    } catch (SQLException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}

//关闭数据库连接（如果采用了连接池，就是归还连接）
public static void releaseConnection(){
    //从threadLocal中获取
    Connection conn = tc.get();
    try {
        if(conn !=null){
            conn.close(); //不是物理关闭，而是放入到连接池中，置为空闲状态
        }
    } catch (Exception e) {
        e.printStackTrace();
    }finally {
        threadLocal.remove();
    }
}
}

```



```
}
```

6、Dbutils

内部的映射，查询时，其结果根据传入的类型的属性，进行返回（所以必须保证属性和sql中字段名字一样，Dbutils内部会自动忽略大小写，以及下划线）

commons-dbutils 是 Apache 组织提供的一个开源 JDBC 工具类库，它是对 JDBC 的简单封装，学习成本极低，并且使用 dbutils 能极大简化 jdbc 编码的工作量，同时也不会影响程序的性能。

其中 QueryRunner 类封装了 SQL 的执行，是线程安全的。

- (1) 可以实现增、删、改、查、批处理、
- (2) 考虑了事务处理需要共用 Connection。
- (3) 该类最主要的就是简单化了 SQL 查询，它与 ResultSetHandler 组合在一起使用可以完成大部分的数据库操作，能够大大减少编码量。
- (4) 不需要手动关闭连接，runner 会自动关闭连接，释放到连接池中

(1) 更新

public int update(Connection conn, String sql, Object... params) throws SQLException: 用来执行一个更新（插入、更新或删除）操作。

.....

(2) 插入

public T insert(Connection conn, String sql, ResultSetHandler rsh, Object... params) throws SQLException: 只支持 INSERT 语句，其中 rsh - The handler used to create the result object from the ResultSet of auto-generated keys. 返回值: An object generated by the handler. 即自动生成的键值

....

(3) 批处理

public int[] batch(Connection conn, String sql, Object[][] params) throws SQLException: INSERT, UPDATE, or DELETE 语句

public T insertBatch(Connection conn, String sql, ResultSetHandler rsh, Object[][] params) throws SQLException: 只支持 INSERT 语句

.....

(4) 使用 QueryRunner 类实现查询

public Object query(Connection conn, String sql, ResultSetHandler rsh, Object... params) throws SQLException: 执行一个查询操作，在这个查询中，对象数组中的每个元素值被用来作为查询语句的置换参数。该方法会自行处理 PreparedStatement 和 ResultSet 的创建和关闭。

....

ResultSetHandler接口用于处理 java.sql.ResultSet，将数据按要求转换为另一种形式。

ResultSetHandler 接口提供了一个单独的方法：Object handle (java.sql.ResultSet rs)该方法的返回值将作为QueryRunner类的query()方法的返回值。

该接口有如下实现类可以使用：

- BeanHandler：将结果集中的第一行数据封装到一个对应的JavaBean实例中。
- BeanListHandler：将结果集中的每一行数据都封装到一个对应的JavaBean实例中，存放到List里。
- ScalarHandler：查询单个值对象
- MapHandler：将结果集中的第一行数据封装到一个Map里，key是列名，value就是对应的值。
- MapListHandler：将结果集中的每一行数据都封装到一个Map里，然后再存放到List
- ColumnListHandler：将结果集中某一列的数据存放到List中。
- KeyedHandler(name)：将结果集中的每一行数据都封装到一个Map里，再把这些map再存到一个map里，其key为指定的key。
- ArrayHandler：把结果集中的第一行数据转成对象数组。
- ArrayListHandler：把结果集中的每一行数据都转成一个数组，再存放到List中。

6.1 使用Dbutils组件封装BaseDAOImpl

```
public class Book {
    private Integer id;
    private String name;
    private String writer;
    private String publish;
    private Integer num;

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getWriter() {
        return writer;
    }

    public void setWriter(String writer) {
        this.writer = writer;
    }

    public String getPublish() {
```

```

        return publish;
    }

    public void setPublish(String publish) {
        this.publish = publish;
    }

    public Integer getNum() {
        return num;
    }

    public void setNum(Integer num) {
        this.num = num;
    }

    @Override
    public String toString() {
        return "Book{" +
            "id=" + id +
            ", name='" + name + '\'' +
            ", writer='" + writer + '\'' +
            ", publish='" + publish + '\'' +
            ", num=" + num +
            '}';
    }
}

```

```

/*

```

如何使用DBUtils?

(1) 导入jar

commons-dbutils-1.6.jar

druid-1.1.10.jar

mysql-connector-java-5.1.36-bin.jar

(2) QueryRunner有各种执行sql方法

public int update(Connection conn, String sql, Object... params) throws

SQLException: 用来执行一个更新（插入、更新或删除）操作。

public Object query(Connection conn, String sql, ResultSetHandler rsh, Object... params) throws SQLException: 执行一个查询操作，在这个查询中，对象数组中的每个元素值被用来作为查询语句的置换参数。该方法会自行处理 PreparedStatement 和 ResultSet 的创建和关闭。

查询的方法query方法，需要配合ResultSetHandler接口的实现类使用。

ResultSetHandler接口有很多实现类:

A: BeanListHandler: 将结果集中的每一行数据都封装到一个对应的JavaBean实例中，存放到List里。

B: BeanHandler: 将结果集中的第一行数据封装到一个对应的JavaBean实例中

C: ScalarHandler: 查询单个值对象

```

*/

```

```

public class TestDBUtils {

```

```

@Test
public void addBook() {
    Book book = new Book();
    book.setName("西游记");
    book.setWriter("吴承恩");
    book.setPublish("大唐出版社");
    book.setNum(100);

    try {

        //1.创建queryRunner对象
        QueryRunner queryRunner=new QueryRunner();

        //2.准备sql
        String sql="insert into book(name,writer,publish,num)
values(?,?,?,?)";

        //3.执行插入操作
        queryRunner.update(JdbcUtils.getConnection(), sql,
book.getName(),book.getWriter(),book.getPublish(),book.getNum());

    }catch (Exception e) {
        e.printStackTrace();
        throw new RuntimeException("修改失败!");
    }
}

@Test
public void updateBook() {

    Book book = new Book();
    book.setId(1);
    book.setName("西游记1");
    book.setWriter("吴承恩2");
    book.setPublish("大唐出版社1");
    book.setNum(100);
    try {

        //1.创建queryRunner对象
        QueryRunner queryRunner=new QueryRunner();

        //2.准备sql
        String sql="update book set name=?,writer=?,publish=?,num=? where
id=?";

        //3.执行插入操作
        queryRunner.update(JdbcUtils.getConnection(), sql,
book.getName(),book.getWriter(),book.getPublish(),book.getNum(),book.getId());

    }catch (Exception e) {
        e.printStackTrace();
        throw new RuntimeException("添加失败!");
    }
}

```

```

@Test
public void findBookById() {

    Integer id = 1;

    try {

        //1.创建QueryRunner对象
        QueryRunner queryRunner=new QueryRunner();

        //2.准备sql
        String sql="select id,name,writer,publish,num from book where id=?";

        //3.执行插入操作
        //BeanListHandler封装多个指定javabean的结果
        Book book = queryRunner.query(JdbcUtils.getConnection(), sql, new
        BeanHandler<Book>(Book.class),id);

        System.out.println(book);

    }catch (Exception e) {
        e.printStackTrace();
        throw new RuntimeException("查询失败!");
    }
}

@Test
public void findBookBywhere() {
    Book book = new Book();

    try {
        //1.创建QueryRunner对象
        QueryRunner queryRunner=new QueryRunner();

        //2.准备sql
        //select id,name,writer,publish,num from book where 1=1 方式2
        StringBuilder sb=new StringBuilder("select
        id,name,writer,publish,num from book where 1=1 ");

        //数据库的模糊查询
        if(StringUtils.isEmpty(book.getName())) {
            sb.append("and name like '%"+book.getName()+"%'");
        }

        if(StringUtils.isEmpty(book.getWriter())) {
            sb.append("and writer like '%"+book.getWriter()+"%'");
        }

        if(StringUtils.isEmpty(book.getPublish())) {
            sb.append("and publish like '%"+book.getPublish()+"%'");
        }
    }
}

```

```

        //3.执行插入操作
        //BeanListHandler封装多个指定javabean的结果
        List<Book> books = queryRunner.query(JdbcUtils.getConnection(),
sb.toString(), new BeanListHandler<Book>(Book.class));
        System.out.println(books);

    }catch (Exception e) {
        e.printStackTrace();
        throw new RuntimeException("查询失败!");
    }
}

@Test
public void deleteBookById() {
    Integer id = 1;
    try {

        //1.创建queryRunner对象
        QueryRunner queryRunner=new QueryRunner();

        //2.准备sql
        String sql="delete from book where id=?";

        //3.执行插入操作
        queryRunner.update(JdbcUtils.getConnection(), sql, id);

    }catch (Exception e) {
        e.printStackTrace();
        throw new RuntimeException("删除失败!");
    }
}
}

```

封装:

```

public abstract class BaseDao<T> {
    private QueryRunner queryRunner = new QueryRunner();

    /**
     * 通用的增删改的方法
     * @param sql String 要执行的sql
     * @param args Object... 如果sql中有?，就传入对应个数的? 要设置值
     * @return int 执行的结果
     */
    protected int update(String sql,Object... args) {
        try {
            return queryRunner.update(JDBCTools.getConnection(),sql,args);
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}

/**

```

```

    * 查询单个对象的方法
    * @param clazz Class 记录对应的类类型
    * @param sql String 查询语句
    * @param args Object... 如果sql中有?，即根据条件查询，可以设置?的值
    * @param <T> 泛型方法声明的泛型类型
    * @return T 一个对象
    */
    protected <T> T getBean(Class<T> clazz, String sql, Object... args){
        List<T> list = getList(clazz, sql, args);
        if(list != null && list.size()>0) {
            return getList(clazz, sql, args).get(0);
        }
        return null;
    }

    /**
    * 通用查询多个对象的方法
    * @param clazz Class 记录对应的类类型
    * @param sql String 查询语句
    * @param args Object... 如果sql中有?，即根据条件查询，可以设置?的值
    * @param <T> 泛型方法声明的泛型类型
    * @return List<T> 把多个对象放到了List集合
    */
    protected <T> List<T> getList(Class<T> clazz, String sql, Object... args){
        try {
            return queryRunner.query(JDBCTools.getConnection(),sql,new
BeanListHandler<T>(clazz),args);
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }

    protected Object getValue(String sql,Object... args){
        try {
            return queryRunner.query(JDBCTools.getConnection(),sql,new
ScalarHandler<>(),args);
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}

```

```

public class EmployeeDAOImpl extends BaseDao implements EmployeeDAO {
    @Override
    public boolean addEmployee(Employee employee) {
        String sql = "insert into
t_employee(`eid`,`ename`,`salary`,`commission_pct`,`birthday`,`" +
        "`gender`,`tel`,`email`,`address`,`work_place`,`hiredate`,`job_id`,`mid`,`did`)
" +
        "values(null,?,?,?,?,?,?,?,?,?,?,?,?,?)";//null表示eid是自增的
        return update(sql,employee.getEname(),
            employee.getSalary(),
            employee.getCommissionPct(),

```

```

        employee.getBirthDay(),
        employee.getGender(),
        employee.getTel(),
        employee.getEmail(),
        employee.getAddress(),
        employee.getWorkPlace(),
        employee.getHiredate(),
        employee.getJobId(),
        employee.getMid(),
        employee.getDid()

    )>0;
}

@Override
public Employee getByEid(int eid) {
//    String sql = "select * from t_employee where eid = ?";
    String sql = "SELECT `eid`,`ename`,`salary`,`commission_pct` AS
commissionPct,`birthday`,`gender`,`tel`,`email`,`address`,`work_place` AS
workPlace,`hiredate`,`job_id` AS jobId,`mid`,`did` FROM t_employee where eid =
?";

    return getBean(Employee.class,sql,eid);
}

@Override
public boolean removeEmployee(int eid) {
    String sql = "delete from t_employee where eid = ?";
    return update(sql,eid)>0;
}

@Override
public List<Employee> getAll() {
//    String sql = "select * from t_employee";
    String sql = "SELECT `eid`,`ename`,`salary`,`commission_pct` AS
commissionPct,`birthday`,`gender`,`tel`,`email`,`address`,`work_place` AS
workPlace,`hiredate`,`job_id` AS jobId,`mid`,`did` FROM t_employee";
    return getList(Employee.class,sql);
}

@Override
public long getEmployeeCount() {
    String sql = "select count(*) from t_employee";
    return (Long) getValue(sql);//Object不能直接强转为long, 只能转为Long包装类, 然
后在自动拆箱
}
}

```

7、sql注入问题

- sql语句使用字符串的拼接, 会导致sql注入, 即传入的参数时, 传入一个sql语句
- 解决方式之一: 对于用户的输入进行检查, (是否有特殊的语句)

```

public class TestSQLInject {
    public static void main(String[] args) throws Exception {
        Scanner input = new Scanner(System.in);
    }
}

```



```

System.out.print("请输入你要查询的员工的编号：");
String id = input.nextLine();

Class.forName("com.mysql.cj.jdbc.Driver");
//B: 获取数据库连接对象
String url = "jdbc:mysql://localhost:3306/aa?serverTimezone=UTC";
Connection connection = DriverManager.getConnection(url, "root",
"root");

String sql = "select * from emp where empno = " + id;
System.out.println("sql = " + sql); //select * from emp where empno = 1
or 1=1

PreparedStatement pst = connection.prepareStatement(sql);

//执行查询
ResultSet rs = pst.executeQuery();
while (rs.next()) {
    for (int i = 1; i < 9; i++) {
        System.out.print(rs.getObject(i) + "\t");
    }
    System.out.println();
}

rs.close();
pst.close();
connection.close();
input.close();
}
}

```

输入id为 1 or 1=1时，
就是sql注入，此时会将
数据表中的数据全部展
示出来

8、获取自增长键值

```

public class TestAutoIncrement {

    @Test
    public void test03() throws Exception{

        //A:把驱动类加载到内存中
        Class.forName("com.mysql.cj.jdbc.Driver");
        //B: 获取数据库连接对象
        String url = "jdbc:mysql://localhost:3306/aa?serverTimezone=UTC";
        Connection connection = DriverManager.getConnection(url, "root", "root");

        String sql = "INSERT INTO t_account(balance)VALUES(?)";

        PreparedStatement pst = connection.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS); //此时对sql进行预编译，里面是带? 的
//Statement.RETURN_GENERATED_KEYS表示，执行sql后，返回自增长键值

        pst.setObject(1,2000);

        //D: 执行sql
        int len = pst.executeUpdate();
    }
}

```

自增长键：该字段不需
要传入，自动生成值，
且该值会自动增加1

使用delete清空表，自
增长值会在清空表之前
的最后一个值的基础上+
1
使用truncate清空表，
自增长值会从1开始

```

        System.out.println(len>0 ? "添加成功" : "添加失败");

        //拿到 自动增长的值是多少
        ResultSet rs = pst.getGeneratedKeys();
        while (rs.next()){
            Object object = rs.getObject(1);
            System.out.println(object);
        }
        pst.close();
        connection.close();
    }
}

```

9、批处理

批量导入数据：先将索引删除或禁用，导完之后在业务低峰的时候再建索引，如果有外键，也是需要删除。

url中开启批处理(?useSSL=false&serverTimezone=Asia/Shanghai&rewriteBatchedStatements=true)
先批量导入到暂存方法中，最后统一执行sql语句
使用多线程

已经有一张表了，我们就想在创建一张一样的表（数据备份）

```
CREATE TABLE t_user_bak AS SELECT * FROM t_user
```

已经有一张表了，我们就想在创建一张一样的表，只要表结构不要数据

```
CREATE TABLE t_user_bak AS SELECT * FROM t_user WHERE 1=2
```



```

public class TestBatch {
    @Test
    public void test01() throws Exception {
        long start = System.currentTimeMillis();

        //把驱动类加载到内存中
        Class.forName("com.mysql.cj.jdbc.Driver");
        //B: 获取数据库连接对象
        String url = "jdbc:mysql://localhost:3306/aa?serverTimezone=UTC";
        Connection connection = DriverManager.getConnection(url, "root",
"root");

        String sql = "insert into t_user_bak(id,username,pwd) values(?,?,?)";
        PreparedStatement pst = connection.prepareStatement(sql);

        for(int i=1; i<=1000000; i++){
            pst.setObject(1,i);
            pst.setObject(2,"username"+i);
            pst.setObject(3,"pwd"+i);

            pst.executeUpdate();//这里就不接收返回值了
        }
    }
}

```

```

        pst.close();
        connection.close();

        long end = System.currentTimeMillis();
        System.out.println("耗时: " + (end-start)); //耗时: 4700
    }

    @Test
    public void test02() throws Exception {
        long start = System.currentTimeMillis();

        //把驱动类加载到内存中
        Class.forName("com.mysql.cj.jdbc.Driver");
        //B: 获取数据库连接对象
        String url = "jdbc:mysql://localhost:3306/aa?serverTimezone=UTC";
        Connection connection = DriverManager.getConnection(url, "root",
"root");

        String sql = "insert into t_user_bak(id,username,pwd) values(?,?,?)";
        PreparedStatement pst = connection.prepareStatement(sql);

        for(int i=1; i<=1000000; i++){
            pst.setObject(1,i);
            pst.setObject(2,"username"+i);
            pst.setObject(3,"pwd"+i);

            pst.addBatch(); //先攒着这些数据，设置完?，sql会重新编译一下，生成一条新的完整
的sql
        }
        pst.executeBatch(); //最后一口气执行

        //          这里虽然调用了批处理执行sql的方法，但是url中没有告诉mysql开启批处理的功能，仍然是一
        条一条添加的

        pst.close();
        connection.close();

        long end = System.currentTimeMillis();
        System.out.println("耗时: " + (end-start)); //耗时: 4714
    }

    @Test
    public void test03() throws Exception {
        long start = System.currentTimeMillis();

        //把驱动类加载到内存中
        Class.forName("com.mysql.cj.jdbc.Driver");
        //B: 获取数据库连接对象
        String url = "jdbc:mysql://localhost:3306/aa?
serverTimezone=UTC&rewriteBatchedStatements=true";
        Connection connection = DriverManager.getConnection(url, "root",
"root");

```

```

String sql = "insert into t_user_bak(id,username,pwd) values(?,?,?)";
PreparedStatement pst = connection.prepareStatement(sql);

for(int i=1; i<=1000000; i++){
    pst.setObject(1,i);
    pst.setObject(2,"username"+i);
    pst.setObject(3,"pwd"+i);

    pst.addBatch();//先攒着这些数据，设置完?，sql会重新编译一下，生成一条新的完整的sql
}
pst.executeBatch();//最后一口气执行

pst.close();
connection.close();

long end = System.currentTimeMillis();
System.out.println("耗时: " + (end-start));//耗时: 1625
}
}

```

使用easyexcel 工具包 导入 据到mysql中